



[Auto]Stitching Photo Mosaics

OVERVIEW

Project 3: [Auto]Stitching Photo Mosaics

This project is all about stitching together many photographs to create larger composite images. The project involves shooting photographs with projective transformations, recovering homographies between image pairs, warping images for alignment, and blending them into seamless mosaics.

The project is divided into four main tasks:

- **A.1: Shoot the Pictures** - Capture photographs with projective transformations
- **A.2: Recover Homographies** - Compute transformation matrices between image pairs
- **A.3: Warp the Images** - Implement image warping with different interpolation methods
- **A.4: Blend the Images into a Mosaic** - Create seamless composite images

TASK A.1

Shoot the Pictures

Shoot two or more photographs so that the transforms between them are projective (a.k.a. perspective). The most common way is to fix the center of projection (COP) and rotate your camera while capturing photos.

Deliverables:

Show at least 2 sets of images with projective transformations between them (fixed center of projection, rotate camera).



Left image



Middle left image



Middle right image

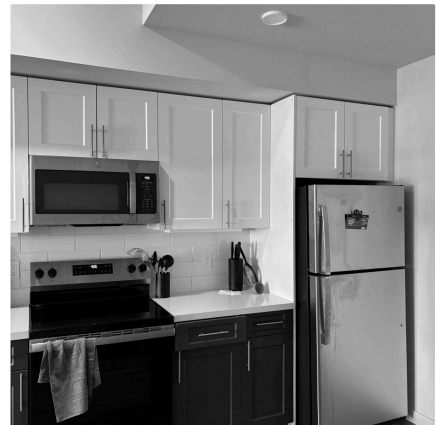
Image set with projective transformations (fixed center of projection, rotated camera)



Kitchen left image



Kitchen middle image

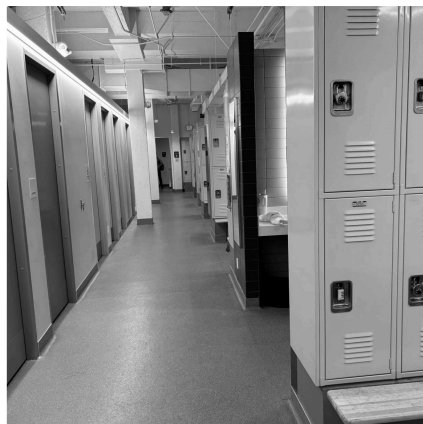


Kitchen right image

Kitchen image set with projective transformations (fixed center of projection, rotated camera)



Locker left image



Locker middle image



Locker right image

TASK A.2

Recover Homographies

Before you can warp your images into alignment, you need to recover the parameters of the transformation between each pair of images. In our case, the transformation is a homography: $\mathbf{p}' = \mathbf{H}\mathbf{p}$, where $\mathbf{H}$ is a 3×3 matrix with 8 degrees of freedom.

Display system of equations

For one correspondence pair $(x,y) \rightarrow (u,v)$, the system of equations is:

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -ux & -uy \\ 0 & 0 & 0 & x & y & 1 & -vx & -vy \end{bmatrix} \times \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix}$$

Where:

- $\mathbf{a}_1^T = [x \ y \ 1 \ 0 \ 0 \ 0 \ -ux \ -uy]$
- $\mathbf{a}_2^T = [0 \ 0 \ 0 \ x \ y \ 1 \ -vx \ -vy]$
- $\mathbf{b}_1 = u$
- $\mathbf{b}_2 = v$

This gives us 2 equations for 8 unknowns. With n correspondence pairs, we get $2n$ equations, forming an overdetermined system that can be solved using least squares when $n \geq 4$.

This is exactly what we discussed in class. The total system of equations can be found below in python code.

Implement computeH(im1_pts, im2_pts) function

Write a function of the form:

```
H = computeH(im1_pts, im2_pts)
```

where `im1_pts` and `im2_pts` are n -by-2 matrices holding the (x,y) locations of n point correspondences from the two images and `H` is the recovered 3×3 homography matrix.

Implementation: The function constructs the matrix A ($2n \times 8$) and vector b ($2n \times 1$) as shown above, then uses least squares to solve $Ah=b$. The solution h contains the 8 homography parameters, which are reshaped into the 3×3 matrix H .

```
# Construct matrix A using numpy indexing
A[0::2, 0:3] = np.stack([x, y, np.ones(n)], axis=1) # First row of each pair
A[1::2, 3:6] = np.stack([x, y, np.ones(n)], axis=1) # Second row of each pair
A[0::2, 6]   = -u * x                               # -ux terms
A[0::2, 7]   = -u * y                               # -uy terms
A[1::2, 6]   = -v * x                               # -vx terms
A[1::2, 7]   = -v * y                               # -vy terms

# Construct vector b
b[0::2] = u # u coordinates
b[1::2] = v # v coordinates

# Solve using least squares
h, residuals, rank, s = np.linalg.lstsq(A, b, rcond=None)
```

Show correspondences visualized on the images

The 8 correspondences (manually selected with ginput) are displayed as red dots on the images for the first and second image as an example.



Left image



Middle left image

Image set with projective transformations (fixed center of projection, rotated camera)

Show recovered homography matrix

The computed 3x3 homography matrix H for the first and second example images is shown below.

$$H = \begin{bmatrix} 2.11756002e+00 & -1.50286932e-02 & -5.99626144e+03 \\ 7.69110234e-01 & 1.90987889e+00 & -2.61261782e+03 \\ 2.63190503e-04 & -5.76557477e-06 & 1.00000000e+00 \end{bmatrix}$$

TASK A.3

Warp the Images

Implement `warpImageNearestNeighbor(im, H)` and `warpImageBilinear(im, H)` using inverse warping

The two different interpolation methods differ in how they pick the pixel values for the output image. Nearest Neighbor Interpolation rounds the coordinates to the nearest pixel value, while Bilinear Interpolation uses a weighted average of the four neighboring pixels.

In code, this difference is between these lines:

```
xi = np.rint(x).astype(np.int64)
yi = np.rint(y).astype(np.int64)

flat_out[inbounds, :] = im[yi[inbounds], xi[inbounds], :]
```

and

```
x0 = np.floor(x[inbounds]).astype(np.int64)
y0 = np.floor(y[inbounds]).astype(np.int64)
x1 = np.minimum(x0 + 1, w_in - 1)
y1 = np.minimum(y0 + 1, h_in - 1)

wa = (x1 - x[inbounds]) * (y1 - y[inbounds])
wb = (x[inbounds] - x0) * (y1 - y[inbounds])
wc = (x1 - x[inbounds]) * (y[inbounds] - y0)
wd = (x[inbounds] - x0) * (y[inbounds] - y0)

flat_out[inbounds, :] = (wa[:, np.newaxis] * im[y0, x0, :] +
                        wb[:, np.newaxis] * im[y0, x1, :] +
                        wc[:, np.newaxis] * im[y1, x0, :] +
                        wd[:, np.newaxis] * im[y1, x1, :])
```

Apply to 2+ images for rectification

I have taken two pictures of rectangular objects and applied the homography to the images to rectify them.



Cal ID



Kitchen drawer

Show comparisons between interpolation methods



Comparisons between interpolation methods

Nearest Neighbor (9.4948s)



Bilinear Interpolation (27.8857s)





Discuss trade-offs between speed and quality

Nearest neighbor interpolation is significantly faster than bilinear interpolation, as shown by the timing results above (9.49s vs. 27.88s, i.e. nearest neighbor is 2.94x faster). On a large scale, both methods produce similar-looking results, and the differences may not be immediately obvious. However, when you zoom in (e.g. on the twigs and rails in the zoomed-in comparison), the **quality improvement** from bilinear interpolation becomes quite noticeable: edges are smoother and there are fewer blocky artifacts compared to nearest neighbor. This makes bilinear interpolation preferable when image quality is important, especially in areas with fine details, while nearest neighbor is a good choice when speed is the priority.

TASK A.4

Blend the Images into a Mosaic

Here are the three source images used for creating the mosaic:



Left Image



Middle Image



Right Image

The final blended mosaic combining all three images:



Final Mosaic: Left + Middle + Right

Blending Process

The images were blended using homography transformations and weighted averaging (0.5 weight for each image) in overlapping regions.

Kitchen Mosaic Example

Here's another example using kitchen images to demonstrate the same blending technique:



Left Image



Middle Image



Right Image

The final blended kitchen mosaic:



Final Kitchen Mosaic: Left + Middle + Right

Locker Room Mosaic Example

Here's another example using locker room images to demonstrate the same blending technique:



Left Image



Middle Image



Right Image

The final blended locker room mosaic:



Final Locker Room Mosaic: Left + Middle + Right

© 2025 — Konstantin Wehmeyer